

BẢN IN MÃ NGUỒN TIÊU BIỂU

Nộp kèm hồ sơ đăng ký quyền tác giả — chương trình máy tính

EnQuiz: Ứng dụng ôn thi trắc nghiệm Khởi sự doanh nghiệp

Đồng tác giả: Đỗ Thùy Hương · Phan Anh Tú

Kho mã nguồn: <https://github.com/thuyhuongctu/EnQuiz>

Bản đóng băng (commit): 8723b1ede171ae1fabf016caba321a314efc40f7

Thời điểm đóng băng: 08/9/2026, 00:00:22 (giờ Việt Nam)

Nội dung: 1162 dòng mã nguồn nguyên văn từ 04 tệp cốt lõi

Ba tệp được in và mã băm SHA-256 (tính tại đúng commit đóng băng):

assets/js/parser.js (284 dòng)

SHA-256: 25675c67296a85156648fa5c98c65487a061b7f79d41610c20977980d7c9df73

assets/js/bank.js (309 dòng)

SHA-256: 6e5a6945c430b7a7e34f9787b6cb5027f568c4246e2ab707653d2e2fcba1a8b9

assets/js/storage.js (236 dòng)

SHA-256: aa6ec25ede9f2308f2553448fb5e8da9cca1ac9ecd843f84e38519e9499dfb88

assets/js/tour.js (333 dòng)

SHA-256: dbff3f843430c3fa14fa176b0bcd65767b2b493b573234e52c57234ff96992cb

Mỗi trang in có số dòng liên tục theo tệp gốc, số trang và chỗ ký nháy.

Kiểm chứng: git show 8723b1e:<tệp> cho ra đúng nội dung in tại đây.

== TỆP: assets/js/parser.js · 284 dòng · SHA-256: 25675c67296a85156648fa5c98c65487...

```

1 | /* =====
2 |     parser.js – Bóc tách câu hỏi trắc nghiệm từ văn bản thô
3 |     =====
4 |     Nhận hai khối văn bản rời (đề và danh sách đáp án) rồi tự
5 |     ghép lại theo SỐ THỬ TỰ CÂU ghi trong đề – không dựa vào vị
6 |     trí dán, nên có thể dán một đoạn đề bất kỳ (câu 31-60...).
7 |     ===== */
8 | (function (global) {
9 |     'use strict';
10 |
11 |     var LETTERS = ['A', 'B', 'C', 'D', 'E', 'F'];
12 |
13 |     function letterIndex(ch) {
14 |         return LETTERS.indexOf(String(ch).toUpperCase());
15 |     }
16 |
17 |     /** Chuẩn hoá xuống dòng, bỏ ký tự ẩn và khoảng trắng thừa do PDF sinh ra. */
18 |     function cleanup(text) {
19 |         return String(text || '')
20 |             .replace(/\r\n?/g, '\n')
21 |             .replace(/[ -]/g, ' ') // no-break space, zero-width...
22 |             .replace(/['']/g, '"')
23 |             .replace(/[""/g, '"')
24 |             .replace(/[ \t]+/g, ' ')
25 |             .replace(/\n{3,}/g, '\n\n')
26 |             .replace(/^\s*(?:trang|page)\s*d+\s*(?:\s*d+)?\s*$/gim, '') // số trang
27 |             .trim();
28 |     }
29 |
30 |     function tidy(s) {
31 |         return String(s || '')
32 |             .replace(/\s*\n\s*/g, ' ')
33 |             .replace(/\s{2,}/g, ' ')
34 |             .replace(/^\s*\s+---]+/, '')
35 |             .replace(/[\s*]+\$/g, '')
36 |             .trim();
37 |     }
38 |
39 |     /* =====
40 |         1. DANH SÁCH ĐÁP ÁN
41 |         ===== */
42 |
43 |     /** Dòng chỉ gồm các cặp "số + chữ cái", ví dụ "4B 5C 6D" hay "1-A, 2-B". */
44 |     var COMPACT_LINE = /\s*(?:\d{1,3}\s*[.:])\s*---]?[A-Fa-f]\s*[.,;]\s*(?:\s*)+$/;
45 |     var PAIR = /(\d{1,3})\s*[.:])\s*---]?[A-Fa-f])/g;
46 |     /** Dòng dạng "Câu 12: B. nội dung..." – chỉ lấy cặp đầu dòng. */
47 |     var LEADING_PAIR = /\s*(?:câu|cau|question|q)?\s*(\d{1,3})\s*[.:])\s*---]?[A-Fa-f])(?![\p{L}\d])
48 |     /iu;
49 |
50 |     /**
51 |      * Đọc danh sách đáp án.
52 |      * @returns {{map: Object, count: number}} map: số câu -> chỉ số phương án đúng
53 |      */
54 |     function parseAnswerKey(text) {
55 |         var src = cleanup(text);
56 |         var map = {};
57 |         var count = 0;
58 |         if (!src) return { map: map, count: 0 };
59 |         src.split('\n').forEach(function (line) {
60 |             if (!line.trim()) return;

```

```

61 |
62 |     if (COMPACT_LINE.test(line)) {
63 |         // dòng liệt kê nhiều đáp án liền nhau
64 |         PAIR.lastIndex = 0;
65 |         var m;
66 |         while ((m = PAIR.exec(line)) !== null) {
67 |             var n = parseInt(m[1], 10);
68 |             if (map[n] === undefined) { map[n] = letterIndex(m[2]); count++; }
69 |         }
70 |         return;
71 |     }
72 |
73 |     // dòng có kèm nội dung: chỉ tin cặp nằm ở đầu dòng
74 |     var lead = line.match(LEADING_PAIR);
75 |     if (lead) {
76 |         var num = parseInt(lead[1], 10);
77 |         if (map[num] === undefined) { map[num] = letterIndex(lead[2]); count++; }
78 |     }
79 | });
80 |
81 | return { map: map, count: count };
82 | }
83 |
84 | /* =====
85 |     2. TÁCH CÂU HỎI
86 |     ===== */
87 |
88 | /* Đầu câu: "Câu 1:", "Câu 1.", "Bài 1 -", hoặc số trần bắt buộc có dấu ngăn "1." / "1)" */
89 | var Q_START = /(?:^|\n)[ \t]*(?:(:?câu|cau|question|bài|bai)[ \t]*(\d{1,3})[ \t]*[.:)\-\---]?|(\d{1,
90 | 3}))[ \t]*[.:)]?[ \t]*/gi;
91 |
92 | /** Cắt văn bản thành từng khối câu hỏi kèm số thứ tự gốc. */
93 | function splitBlocks(text) {
94 |     var src = cleanup(text);
95 |     var marks = [];
96 |     var m;
97 |
98 |     Q_START.lastIndex = 0;
99 |     while ((m = Q_START.exec(src)) !== null) {
100 |         marks.push({
101 |             number: parseInt(m[1] || m[2], 10),
102 |             start: m.index + m[0].length,
103 |             markStart: m.index
104 |         });
105 |         // cho phép lần lặp kế tiếp bắt đầu ngay sau dấu hiệu vừa tìm được
106 |         Q_START.lastIndex = m.index + m[0].length;
107 |     }
108 |
109 |     if (!marks.length) {
110 |         return src ? [{ number: 1, body: src }] : [];
111 |     }
112 |
113 |     return marks.map(function (mk, i) {
114 |         var end = i + 1 < marks.length ? marks[i + 1].markStart : src.length;
115 |         return { number: mk.number, body: src.slice(mk.start, end) };
116 |     });
117 | }
118 |
119 | /* =====
120 |     3. TÁCH PHƯƠNG ÁN TRONG MỘT KHỐI
121 |     ===== */

```

```

122 | /* Dấu hiệu phương án: đầu dòng hoặc sau khoảng trắng – "A.", "A)", "(A)", "**B." */
123 | var OPT_MARK = /(?:^|\n|[\t])([\\[]?)\s*(\*)\s*([A-Fa-f])\s*([\]\.:\-\-])\s*/g;
124 |
125 | /**
126 |  * Tìm chuỗi phương án A → B → C... theo đúng thứ tự chữ cái.
127 |  * Ràng buộc "tăng dần" giúp loại bỏ các dấu hiệu giả trong nội dung câu hỏi.
128 |  */
129 | function findOptionMarks(body) {
130 |     var candS = [];
131 |     var m;
132 |     OPT_MARK.lastIndex = 0;
133 |     while ((m = OPT_MARK.exec(body)) !== null) {
134 |         candS.push({
135 |             letter: String(m[3]).toUpperCase(),
136 |             starred: m[2] === '*',
137 |             markStart: m.index + (/^[\\n\\t]/.test(m[0]) ? 1 : 0),
138 |             contentStart: m.index + m[0].length
139 |         });
140 |         OPT_MARK.lastIndex = m.index + m[0].length - 1;
141 |     }
142 |
143 |     var chain = [];
144 |     for (var i = 0; i < candS.length; i++) {
145 |         if (candS[i].letter !== 'A') continue;
146 |         chain = [candS[i]];
147 |         var want = 1;
148 |         for (var j = i + 1; j < candS.length && want < LETTERS.length; j++) {
149 |             if (candS[j].letter === LETTERS[want]) {
150 |                 chain.push(candS[j]);
151 |                 want++;
152 |             }
153 |         }
154 |         if (chain.length >= 2) return chain;
155 |     }
156 |     return chain.length >= 2 ? chain : [];
157 | }
158 |
159 | /** Bóc phần "Đáp án: B" và "Giải thích: ..." ra khỏi khối, trả về phần còn lại. */
160 | function extractMeta(body) {
161 |     var answer = -1;
162 |     var explain = '';
163 |
164 |     var ex = body.match(/(?:^|\n)\s*(?:giải\s*thích|giai\s*thich|explanation|lý\s*giải)\s*[:\.\-]\s*(
    □ [\s\S]*)$/i);
165 |     if (ex) {
166 |         explain = tidy(ex[1]);
167 |         body = body.slice(0, ex.index);
168 |     }
169 |
170 |     var an = body.match(/(?:^|\n)\s*(?:đáp\s*án(?:\s*đúng)?|dap\s*an|đ\.\?a|answer|key)\s*[:\.\-]\s*(
    □ [A-Fa-f])(?![\p{L}\d])/iu);
171 |     if (an) {
172 |         answer = letterIndex(an[1]);
173 |         body = body.slice(0, an.index) + body.slice(an.index + an[0].length);
174 |     }
175 |
176 |     return { body: body, answer: answer, explain: explain };
177 | }
178 |
179 | /** Phân tích một khối thành câu hỏi hoàn chỉnh. */
180 | function parseBlock(block) {
181 |     var meta = extractMeta(block.body);

```

```
182 |     var body = meta.body;
183 |     var marks = findOptionMarks(body);
184 |
185 |     var out = {
186 |         number: block.number,
187 |         text: '',
188 |         options: [],
189 |         correct: -1,
190 |         explain: meta.explain,
191 |         warnings: []
192 |     };
193 |
194 |     if (!marks.length) {
195 |         out.text = tidy(body);
196 |         out.warnings.push('Không tìm thấy các phương án A, B, C...');
197 |         return out;
198 |     }
199 |
200 |     out.text = tidy(body.slice(0, marks[0].markStart));
201 |
202 |     marks.forEach(function (mk, i) {
203 |         var end = i + 1 < marks.length ? marks[i + 1].markStart : body.length;
204 |         var raw = body.slice(mk.contentStart, end);
205 |         var starred = mk.starred || /^s*\$/.test(raw);
206 |         out.options.push(tidy(raw));
207 |         if (starred && out.correct === -1) out.correct = i;
208 |     });
209 |
210 |     if (meta.answer !== -1 && meta.answer < out.options.length) out.correct = meta.answer;
211 |
212 |     if (!out.text) out.warnings.push('Thiếu nội dung câu hỏi');
213 |     if (out.options.some(function (o) { return !o; })) out.warnings.push('Có phương án bị rỗng');
214 |
215 |     return out;
216 | }
217 |
218 | /* =====
219 |    4. GHÉP ĐỀ VỚI ĐÁP ÁN
220 |    ===== */
221 |
222 | var Parser = {
223 |     parseAnswerKey: parseAnswerKey,
224 |
225 |     /**
226 |      * @param {string} questionText nội dung đề
227 |      * @param {string} answerText  danh sách đáp án (có thể rỗng)
228 |      * @returns {{questions:Array, stats:Object}}
229 |      */
230 |     parse: function (questionText, answerText) {
231 |         var key = parseAnswerKey(answerText);
232 |         var blocks = splitBlocks(questionText);
233 |         var seenNumbers = {};
234 |         var questions = [];
235 |
236 |         blocks.forEach(function (b, i) {
237 |             var q = parseBlock(b);
238 |
239 |             // số câu trùng nhau (PDF nhiều phần) -> dùng thứ tự xuất hiện
240 |             if (seenNumbers[q.number]) q.number = 0;
241 |             else seenNumbers[q.number] = true;
242 |             if (!q.number) q.number = i + 1;
243 |         });
```

```

244 |         if (q.correct === -1 && key.map[q.number] !== undefined) {
245 |             var idx = key.map[q.number];
246 |             if (idx >= 0 && idx < q.options.length) q.correct = idx;
247 |             else q.warnings.push('Đáp án ' + LETTERS[idx] + ' vượt quá số phương án của câu này');
248 |         }
249 |
250 |         if (q.correct === -1) q.warnings.push('Chưa xác định được đáp án đúng');
251 |         q.valid = q.options.length >= 2 && !!q.text && q.correct !== -1;
252 |         questions.push(q);
253 |     });
254 |
255 |     var valid = questions.filter(function (q) { return q.valid; }).length;
256 |
257 |     return {
258 |         questions: questions,
259 |         stats: {
260 |             detected: questions.length,
261 |             valid: valid,
262 |             invalid: questions.length - valid,
263 |             answerKeys: key.count
264 |         }
265 |     };
266 | },
267 |
268 | /** Chuyển kết quả phân tích sang định dạng gói của QuestionBank. */
269 | toPack: function (questions, chapterId, chapterTitle, order) {
270 |     return {
271 |         id: chapterId,
272 |         title: chapterTitle,
273 |         order: order,
274 |         questions: questions
275 |             .filter(function (q) { return q.valid; })
276 |             .map(function (q) {
277 |                 return { q: q.text, a: q.options, c: q.correct, e: q.explain };
278 |             })
279 |     };
280 | }
281 | };
282 |
283 | global.Parser = Parser;
284 | })(window);

```

== TẬP: assets/js/bank.js · 309 dòng · SHA-256: 6e5a6945c430b7a7e34f9787b6cb5027...

```

1 | /* =====
2 | bank.js – Ngân hàng câu hỏi: nạp, chuẩn hoá và GỘP TỰ ĐỘNG
3 | =====
4 | Mỗi tệp trong thư mục data/ tự đăng ký nội dung của mình
5 | bằng cách gọi QuestionBank.register({...}). Bank sẽ:
6 |     1. chuẩn hoá mọi định dạng câu hỏi về một cấu trúc duy nhất
7 |     2. gộp các chương trùng mã (id) làm một
8 |     3. loại bỏ câu hỏi trùng lặp theo nội dung đã chuẩn hoá
9 |     4. gán uid ổn định cho từng câu để lưu tiến độ
10 | Nhờ vậy, thêm một tệp câu hỏi mới chỉ cần khai báo tên tệp
11 | trong data/manifest.js – không phải sửa mã nguồn ứng dụng.
12 | ===== */
13 | (function (global) {
14 |     'use strict';
15 |
16 |     var LETTERS = ['A', 'B', 'C', 'D', 'E', 'F'];
17 |
18 |     /* ----- Tiện ích ----- */
19 |

```

```
20 | /** Bỏ dấu tiếng Việt, hạ chữ thường, gom khoảng trắng – dùng để so trùng. */
21 | function normalizeText(s) {
22 |     return String(s == null ? '' : s)
23 |         .normalize('NFD')
24 |         .replace(/[^\w]/g, '')
25 |         .replace(/đ/gi, 'd')
26 |         .toLowerCase()
27 |         .replace(/[\^p{L}\p{N}]+/gu, ' ')
28 |         .trim();
29 | }
30 |
31 | /** Băm FNV-1a 32 bit -> chuỗi base36, dùng làm uid ổn định giữa các phiên. */
32 | function hash(str) {
33 |     var h = 0x811c9dc5;
34 |     for (var i = 0; i < str.length; i++) {
35 |         h ^= str.charCodeAt(i);
36 |         h = (h + ((h << 1) + (h << 4) + (h << 7) + (h << 8) + (h << 24))) >>> 0;
37 |     }
38 |     return h.toString(36);
39 | }
40 |
41 | /** Xác định chỉ số đáp án đúng từ nhiều kiểu khai báo khác nhau. */
42 | function resolveAnswer(raw, options) {
43 |     if (typeof raw === 'number' && raw >= 0 && raw < options.length) return raw;
44 |
45 |     if (typeof raw === 'string') {
46 |         var t = raw.trim();
47 |         // dạng "A" / "b)" / "C."
48 |         var letter = t.replace(/[\s]/g, '').toUpperCase();
49 |         if (letter.length === 1 && LETTERS.indexOf(letter) !== -1) {
50 |             var li = LETTERS.indexOf(letter);
51 |             if (li < options.length) return li;
52 |         }
53 |         // dạng "1" / "2"
54 |         if (/^\d+$/.test(t)) {
55 |             var n = parseInt(t, 10);
56 |             if (n >= 1 && n <= options.length) return n - 1;
57 |         }
58 |         // dạng trùng nguyên văn nội dung đáp án
59 |         var norm = normalizeText(t);
60 |         for (var i = 0; i < options.length; i++) {
61 |             if (normalizeText(options[i]) === norm) return i;
62 |         }
63 |     }
64 |     return -1;
65 | }
66 |
67 | /* ----- Chuẩn hoá ----- */
68 |
69 | /**
70 |  * Đưa một câu hỏi thô về cấu trúc chuẩn.
71 |  * Chấp nhận các tên trường: q/question/cauHoi, a/options/answers/luaChon,
72 |  * c/correct/answer/dapAn, e/explain/explanation/giaiThich.
73 |  * @returns {object|null} null nếu câu hỏi không hợp lệ.
74 |  */
75 | function normalizeQuestion(raw, chapter, index) {
76 |     if (!raw || typeof raw !== 'object') return null;
77 |
78 |     var text = raw.q || raw.question || raw.cauHoi || raw.text;
79 |     var options = raw.a || raw.options || raw.answers || raw.luaChon || raw.choices;
80 |     if (!text || !Array.isArray(options) || options.length < 2) return null;
81 | }
```

```

82 |     options = options.map(function (o) { return String(o).trim(); })
83 |         .filter(function (o) { return o.length > 0; });
84 |     if (options.length < 2) return null;
85 |
86 |     var rawAnswer = raw.c;
87 |     if (rawAnswer === undefined) rawAnswer = raw.correct;
88 |     if (rawAnswer === undefined) rawAnswer = raw.answer;
89 |     if (rawAnswer === undefined) rawAnswer = raw.dapAn;
90 |
91 |     var correct = resolveAnswer(rawAnswer, options);
92 |     if (correct === -1) return null;
93 |
94 |     var explain = raw.e || raw.explain || raw.explanation || raw.giaiThich || '';
95 |     var key = normalizeText(text);
96 |
97 |     return {
98 |         uid: 'q' + hash(key),
99 |         key: key,
100 |         text: String(text).trim(),
101 |         options: options,
102 |         correct: correct,
103 |         explain: String(explain).trim(),
104 |         chapterId: chapter.id,
105 |         chapterTitle: chapter.title,
106 |         chapterTitleEn: chapter.titleEn || '',
107 |         order: index
108 |     };
109 | }
110 |
111 | /* ----- Bộ máy ngân hàng câu hỏi ----- */
112 |
113 | var chapters = [];           // [{ id, title, order, questions: [] }]
114 | var chapterIndex = {};      // id -> chapter
115 | var questionIndex = {};     // key chuẩn hoá -> question (dùng để chống trùng)
116 | var sources = [];           // nhật ký nạp dữ liệu
117 | var pending = [];           // các gói đăng ký chờ gộp
118 |
119 | function getOrCreateChapter(meta) {
120 |     var id = String(meta.id || 'khac').trim();
121 |     if (!chapterIndex[id]) {
122 |         chapterIndex[id] = {
123 |             id: id,
124 |             title: String(meta.title || id).trim(),
125 |             titleEn: String(meta.titleEn || '').trim(),
126 |             order: typeof meta.order === 'number' ? meta.order : chapters.length + 1,
127 |             questions: []
128 |         };
129 |         chapters.push(chapterIndex[id]);
130 |     } else {
131 |         // bổ sung tiêu đề nếu lần đăng ký trước chưa có
132 |         if (meta.title && chapterIndex[id].title === id) chapterIndex[id].title = String(meta.title).t
133 |         if (meta.titleEn && !chapterIndex[id].titleEn) chapterIndex[id].titleEn = String(meta.titleEn)
134 |     }
135 |     return chapterIndex[id];
136 | }
137 |
138 | /**
139 |  * Gộp một gói câu hỏi vào ngân hàng.
140 |  * @returns {{added:number, duplicated:number, invalid:number}}
141 |  */

```



```
142 | function mergePack(pack, sourceName) {
143 |     var result = { added: 0, duplicated: 0, invalid: 0, chapter: '' };
144 |     if (!pack || typeof pack !== 'object') { result.invalid = 1; return result; }
145 |
146 |     var list = pack.questions || pack.items || pack.data;
147 |     if (!Array.isArray(list)) { result.invalid = 1; return result; }
148 |
149 |     var chapter = getOrCreateChapter(pack);
150 |     result.chapter = chapter.title;
151 |
152 |     list.forEach(function (raw, i) {
153 |         var q = normalizeQuestion(raw, chapter, chapter.questions.length + i);
154 |         if (!q) { result.invalid++; return; }
155 |
156 |         if (questionIndex[q.key]) {
157 |             // Đã có câu hỏi cùng nội dung: bổ sung phần giải thích nếu còn thiếu.
158 |             var existing = questionIndex[q.key];
159 |             if (!existing.explain && q.explain) existing.explain = q.explain;
160 |             result.duplicated++;
161 |             return;
162 |         }
163 |
164 |         questionIndex[q.key] = q;
165 |         chapter.questions.push(q);
166 |         result.added++;
167 |     });
168 |
169 |     sources.push({
170 |         name: sourceName || pack.source || chapter.id,
171 |         chapter: chapter.title,
172 |         added: result.added,
173 |         duplicated: result.duplicated,
174 |         invalid: result.invalid
175 |     });
176 |
177 |     return result;
178 | }
179 |
180 | /** Sắp xếp chương và đánh số thứ tự câu hỏi sau khi gộp xong. */
181 | function finalize() {
182 |     chapters.sort(function (a, b) { return a.order - b.order; });
183 |     chapters.forEach(function (c) {
184 |         c.questions.forEach(function (q, i) { q.order = i + 1; });
185 |     });
186 |     // loại bỏ chương rỗng
187 |     for (var i = chapters.length - 1; i >= 0; i--) {
188 |         if (!chapters[i].questions.length) {
189 |             delete chapterIndex[chapters[i].id];
190 |             chapters.splice(i, 1);
191 |         }
192 |     }
193 | }
194 |
195 | /** Nạp lần lượt các tệp dữ liệu bằng thẻ <script> (chạy được cả khi mở file://). */
196 | function loadScript(src) {
197 |     return new Promise(function (resolve) {
198 |         var el = document.createElement('script');
199 |         el.src = src;
200 |         el.onload = function () { resolve({ src: src, ok: true }); };
201 |         el.onerror = function () { resolve({ src: src, ok: false }); };
202 |         document.head.appendChild(el);
203 |     });
```

```
204 | }
205 |
206 | var QuestionBank = {
207 |     /** Các tệp dữ liệu gọi hàm này để đăng ký nội dung của mình. */
208 |     register: function (pack) {
209 |         pending.push(pack);
210 |         return this;
211 |     },
212 |
213 |     /**
214 |      * Nạp toàn bộ tệp trong manifest rồi gộp lại.
215 |      * @param {string[]} files danh sách đường dẫn tương đối
216 |      */
217 |     loadAll: function (files) {
218 |         var self = this;
219 |         var list = Array.isArray(files) ? files : [];
220 |         var chain = Promise.resolve();
221 |         var failed = [];
222 |
223 |         list.forEach(function (f) {
224 |             chain = chain.then(function () {
225 |                 return loadScript(f).then(function (r) {
226 |                     if (!r.ok) failed.push(f);
227 |                     // gộp ngay các gói mà tệp vừa nạp đã đăng ký
228 |                     while (pending.length) mergePack(pending.shift(), f);
229 |                 });
230 |             });
231 |         });
232 |
233 |         return chain.then(function () {
234 |             while (pending.length) mergePack(pending.shift(), 'inline');
235 |             finalize();
236 |             return { chapters: chapters, failed: failed, total: self.total() };
237 |         });
238 |     },
239 |
240 |     /** Gộp thêm dữ liệu từ chuỗi JSON do người dùng nhập vào. */
241 |     importJSON: function (jsonText, sourceName) {
242 |         var parsed = JSON.parse(jsonText);
243 |         var packs = Array.isArray(parsed) ? parsed : [parsed];
244 |         var total = { added: 0, duplicated: 0, invalid: 0 };
245 |
246 |         packs.forEach(function (p) {
247 |             // cho phép truyền thẳng một mảng câu hỏi không kèm thông tin chương
248 |             if (Array.isArray(p)) p = { id: 'import', title: 'Câu hỏi nhập thêm', questions: p };
249 |             var r = mergePack(p, sourceName || 'import');
250 |             total.added += r.added;
251 |             total.duplicated += r.duplicated;
252 |             total.invalid += r.invalid;
253 |         });
254 |
255 |         finalize();
256 |         return total;
257 |     },
258 |
259 |     /** Gộp trực tiếp một gói đã có sẵn trong bộ nhớ (không qua tệp). */
260 |     merge: function (pack, sourceName) {
261 |         var r = mergePack(pack, sourceName || 'runtime');
262 |         finalize();
263 |         return r;
264 |     },
265 | }
```

```

266 |     /** Gỡ toàn bộ một chương khỏi ngân hàng (dùng khi thay thế bộ đề). */
267 |     removeChapter: function (id) {
268 |         var c = chapterIndex[id];
269 |         if (!c) return 0;
270 |         var n = c.questions.length;
271 |         c.questions.forEach(function (q) { delete questionIndex[q.key]; });
272 |         chapters.splice(chapters.indexOf(c), 1);
273 |         delete chapterIndex[id];
274 |         return n;
275 |     },
276 |
277 |     /* ----- Truy vấn ----- */
278 |     chapters: function () { return chapters; },
279 |
280 |     chapter: function (id) { return chapterIndex[id] || null; },
281 |
282 |     all: function () {
283 |         return chapters.reduce(function (acc, c) { return acc.concat(c.questions); }, []);
284 |     },
285 |
286 |     byUids: function (uids) {
287 |         var set = {};
288 |         this.all().forEach(function (q) { set[q.uid] = q; });
289 |         return (uids || []).map(function (u) { return set[u]; })
290 |             .filter(Boolean);
291 |     },
292 |
293 |     byChapters: function (ids) {
294 |         if (!ids || !ids.length) return this.all();
295 |         return chapters.filter(function (c) { return ids.indexOf(c.id) !== -1; })
296 |             .reduce(function (acc, c) { return acc.concat(c.questions); }, []);
297 |     },
298 |
299 |     total: function () {
300 |         return chapters.reduce(function (n, c) { return n + c.questions.length; }, 0);
301 |     },
302 |
303 |     sources: function () { return sources; }
304 | };
305 |
306 | global.QuestionBank = QuestionBank;
307 | /** Bí danh ngắn gọn cho các tệp dữ liệu. */
308 | global.registerBank = function (pack) { return QuestionBank.register(pack); };
309 | })(window);

```

== Tệp: assets/js/storage.js · 236 dòng · SHA-256: aa6ec25ede9f2308f2553448fb5e8da9...

```

1 | /* =====
2 |     storage.js – lưu tiến độ học tập trong localStorage
3 |     ===== */
4 | (function (global) {
5 |     'use strict';
6 |
7 |     var KEY = 'ksdn.progress.v1';
8 |
9 |     var DEFAULTS = {
10 |         theme: 'auto',    // theo cài đặt sáng/tối của máy, tới khi người dùng tự chọn
11 |         themeMoved: 0,    // đã một lần dời "nền tối" cũ sang Đất sét noir hay chưa
12 |         song: 'vocal',    // bản nhạc đang chọn: 'vocal' có lời hoặc 'instrumental' không lời
13 |         marked: [],       // danh sách uid câu hỏi được đánh dấu
14 |         wrong: [],        // danh sách uid câu hỏi từng trả lời sai
15 |         history: [],      // lịch sử các lần làm bài (tối đa 30 bản ghi)
16 |         // Lịch sử bị cắt còn 30 bản ghi nên không đếm được tổng số lần làm bài

```

```
17 | // hay chuỗi ngày học. Hai mục dưới đây giữ riêng cho phần huy hiệu.
18 | days: [], // các ngày từng làm bài, dạng 'YYYY-MM-DD', không trùng
19 | attempts: 0, // tổng số lần làm bài, không bị cắt bớt
20 | stats: {}, // uid -> { seen, correct }
21 | custom: [] // các bộ đề do người dùng nhập, dạng gói của QuestionBank
22 | };
23 |
24 | /* Ngày theo giờ địa phương; không dùng toISOString vì nó quy về giờ UTC,
25 | làm các buổi học đêm bị tính sang ngày hôm sau. */
26 | function dayKey(ts) {
27 |     var d = new Date(ts);
28 |     return d.getFullYear() + '-' +
29 |         String(d.getMonth() + 1).padStart(2, '0') + '-' +
30 |         String(d.getDate()).padStart(2, '0');
31 | }
32 |
33 | function clone(o) { return JSON.parse(JSON.stringify(o)); }
34 |
35 | var mem = clone(DEFAULTS); // bản sao trong bộ nhớ, dùng khi localStorage bị chặn
36 | var usable = true;
37 |
38 | function readRaw() {
39 |     try {
40 |         var raw = global.localStorage.getItem(KEY);
41 |         return raw ? JSON.parse(raw) : null;
42 |     } catch (e) {
43 |         usable = false;
44 |         return null;
45 |     }
46 | }
47 |
48 | function writeRaw(data) {
49 |     if (!usable) return;
50 |     try {
51 |         global.localStorage.setItem(KEY, JSON.stringify(data));
52 |     } catch (e) {
53 |         usable = false;
54 |     }
55 | }
56 |
57 | function load() {
58 |     var saved = readRaw();
59 |     mem = clone(DEFAULTS);
60 |     if (saved && typeof saved === 'object') {
61 |         Object.keys(DEFAULTS).forEach(function (k) {
62 |             if (saved[k] !== undefined && saved[k] !== null) mem[k] = saved[k];
63 |         });
64 |     }
65 |     return mem;
66 | }
67 |
68 | var Store = {
69 |     /** Toàn bộ dữ liệu hiện tại. */
70 |     get data() { return mem; },
71 |
72 |     /** localStorage có dùng được không (chế độ riêng tư có thể chặn). */
73 |     get available() { return usable; },
74 |
75 |     init: function () { load(); return mem; },
76 |
77 |     save: function () { writeRaw(mem); },
78 | }
```

```
79 |     get: function (key) { return mem[key]; },
80 |
81 |     set: function (key, value) {
82 |         mem[key] = value;
83 |         this.save();
84 |     },
85 |
86 |     /* ----- Đánh dấu câu hỏi ----- */
87 |     isMarked: function (uid) { return mem.marked.indexOf(uid) !== -1; },
88 |
89 |     toggleMark: function (uid) {
90 |         var i = mem.marked.indexOf(uid);
91 |         if (i === -1) mem.marked.push(uid); else mem.marked.splice(i, 1);
92 |         this.save();
93 |         return i === -1;
94 |     },
95 |
96 |     /* ----- Câu trả lời sai ----- */
97 |     addWrong: function (uid) {
98 |         if (mem.wrong.indexOf(uid) === -1) mem.wrong.push(uid);
99 |     },
100 |
101 |     removeWrong: function (uid) {
102 |         var i = mem.wrong.indexOf(uid);
103 |         if (i !== -1) mem.wrong.splice(i, 1);
104 |     },
105 |
106 |     /* ----- Thống kê từng câu ----- */
107 |     recordAnswer: function (uid, isCorrect) {
108 |         var s = mem.stats[uid] || { seen: 0, correct: 0 };
109 |         s.seen += 1;
110 |         if (isCorrect) {
111 |             s.correct += 1;
112 |             this.removeWrong(uid);
113 |         } else {
114 |             this.addWrong(uid);
115 |         }
116 |         mem.stats[uid] = s;
117 |     },
118 |
119 |     /* ----- Lịch sử làm bài ----- */
120 |     addHistory: function (entry) {
121 |         mem.history.unshift(entry);
122 |         if (mem.history.length > 30) mem.history.length = 30;
123 |
124 |         mem.attempts = (mem.attempts || 0) + 1;
125 |         var key = dayKey(entry.at || Date.now());
126 |         if (!mem.days) mem.days = [];
127 |         if (mem.days.indexOf(key) === -1) {
128 |             mem.days.push(key);
129 |             if (mem.days.length > 400) mem.days.shift();
130 |         }
131 |         this.save();
132 |     },
133 |
134 |     clearHistory: function () {
135 |         // Xoá luôn dữ liệu huy hiệu: để lại huy hiệu "50 đề" trong khi lịch sử
136 |         // trống rỗng thì người học không hiểu con số ở đâu ra.
137 |         mem.history = [];
138 |         mem.days = [];
139 |         mem.attempts = 0;
140 |         this.save();
```

```
141 | },
142 |
143 | /** Số ngày học liên tiếp tính đến hôm nay (hôm qua vẫn được tính tiếp). */
144 | streak: function () {
145 |     var days = mem.days || [];
146 |     if (!days.length) return 0;
147 |     var set = {};
148 |     days.forEach(function (d) { set[d] = true; });
149 |
150 |     var probe = new Date();
151 |     if (!set[dayKey(probe)]) {
152 |         // chưa học hôm nay thì chuỗi vẫn còn hiệu lực nếu hôm qua có học
153 |         probe.setDate(probe.getDate() - 1);
154 |         if (!set[dayKey(probe)]) return 0;
155 |     }
156 |     var n = 0;
157 |     while (set[dayKey(probe)]) {
158 |         n++;
159 |         probe.setDate(probe.getDate() - 1);
160 |     }
161 |     return n;
162 | },
163 |
164 | /** Tổng số lần làm bài, kể cả những lần đã rơi khỏi lịch sử 30 bản ghi. */
165 | attemptCount: function () {
166 |     return mem.attempts || mem.history.length;
167 | },
168 |
169 | bestScore: function () {
170 |     if (!mem.history.length) return null;
171 |     return mem.history.reduce(function (max, h) {
172 |         return h.score > max ? h.score : max;
173 |     }, 0);
174 | },
175 |
176 | /* ----- Bộ đề người dùng tự nhập ----- */
177 | customPack: function (id) {
178 |     for (var i = 0; i < mem.custom.length; i++) {
179 |         if (mem.custom[i].id === id) return mem.custom[i];
180 |     }
181 |     return null;
182 | },
183 |
184 | /**
185 |  * Lưu một bộ đề tự nhập.
186 |  * @param {object} pack gói câu hỏi
187 |  * @param {boolean} replace true = ghi đè, false = nối thêm
188 |  */
189 | saveCustom: function (pack, replace) {
190 |     var existing = this.customPack(pack.id);
191 |     if (existing && !replace) {
192 |         existing.title = pack.title || existing.title;
193 |         existing.questions = existing.questions.concat(pack.questions);
194 |     } else if (existing) {
195 |         existing.title = pack.title || existing.title;
196 |         existing.questions = pack.questions.slice();
197 |     } else {
198 |         mem.custom.push({
199 |             id: pack.id,
200 |             title: pack.title,
201 |             order: pack.order,
202 |             questions: pack.questions.slice()
```

```

203 |     });
204 |     }
205 |     this.save();
206 |     return this.customPack(pack.id);
207 | },
208 |
209 |     removeCustom: function (id) {
210 |         for (var i = 0; i < mem.custom.length; i++) {
211 |             if (mem.custom[i].id === id) {
212 |                 mem.custom.splice(i, 1);
213 |                 this.save();
214 |                 return true;
215 |             }
216 |         }
217 |         return false;
218 |     },
219 |
220 |     clearCustom: function () {
221 |         mem.custom = [];
222 |         this.save();
223 |     },
224 |
225 |     reset: function () {
226 |         mem = clone(DEFAULTS);
227 |         try { global.localStorage.removeItem(KEY); } catch (e) { /* bỏ qua */ }
228 |     },
229 |
230 |     exportJSON: function () {
231 |         return JSON.stringify(mem, null, 2);
232 |     }
233 | };
234 |
235 | global.Store = Store;
236 | })(window);

```

== TỆP: assets/js/tour.js · 333 dòng · SHA-256: dbff3f843430c3fa14fa176b0bcd6576...

```

1 | /* =====
2 |     tour.js - "Hương AI": tour hướng dẫn có giọng đọc
3 |     -----
4 |     Dùng Web Speech API sẵn có của trình duyệt nên không cần
5 |     tệp âm thanh: giọng đọc tự đổi theo ngôn ngữ đang chọn.
6 |     Máy nào không có giọng đọc thì phần lời vẫn hiện thành chữ,
7 |     nên tour luôn dùng được.
8 |     ===== */
9 | (function (global) {
10 |     'use strict';
11 |
12 |     var $ = function (sel) { return document.querySelector(sel); };
13 |     var t = function (key, vars) { return global.I18n.t(key, vars); };
14 |
15 |     /* Mỗi bước gắn với một vùng trên trang; thiếu vùng thì bước đó được bỏ qua.
16 |        `pose` chọn tư thế nhân vật đứng cạnh bảng thoại. */
17 |     var STEPS = [
18 |         // Thẻ giảng viên đã gộp vào khối đầu trang, nên ba bước mở đầu trở vào
19 |         // ba phần khác nhau của khối ấy thay vì trở chung một thẻ.
20 |         { key: 'tour.s0', target: '.hero__mascot', lang: 'fr', pose: 'welcome' }, // lời chào tiếng Pháp
21 |         { key: 'tour.s1', target: '.hero', pose: 'welcome' },
22 |         { key: 'tour.s2', target: '.hero__note', pose: 'stand' },
23 |         { key: 'tour.s3', target: '.modes', pose: 'point' },
24 |         { key: 'tour.s4', target: '.modes [data-mode="import"]', pose: 'tablet' },
25 |         { key: 'tour.s5', target: '#chapterList', pose: 'point' },

```

```
26 | // Chỉ một trong hai bước dưới đây hiện ra: thanh dọc có ở màn rộng,
27 | // thanh dưới có ở màn hẹp; bước nào không nhìn thấy thì bị lọc bỏ.
28 | { key: 'tour.nav', target: '.sidenav', pose: 'point' },
29 | { key: 'tour.navm', target: '.botnav', pose: 'point' },
30 | { key: 'tour.s6', target: '.topbar__actions', pose: 'tablet' },
31 | { key: 'tour.s7', target: '.footer', pose: 'welcome' },
32 | // Bước khép lại: Hương AI tự giới thiệu và chỉ cách gọi mình quay lại.
33 | // Trỏ vào bong bóng thoại đầu trang chứ không vào nút gọi tour, vì nút ấy
34 | // bị ẩn suốt lúc tour chạy nên sẽ bị lọc mất.
35 | { key: 'tour.s8', target: '.hero__bubble', pose: 'cheer' }
36 | ];
37 |
38 | /* Người dẫn tour mặc áo len, khác với ảnh áo dài dùng ở thẻ giảng viên và
39 | màn hình kết quả: tour là lúc trò chuyện hướng dẫn nên bộ dáng đời thường
40 | hợp hơn, và cũng để người học phân biệt ngay đầu là lời dẫn. */
41 | var POSE_SRC = {
42 |   welcome: 'assets/img/vest-welcome.webp',
43 |   stand: 'assets/img/vest-stand.webp',
44 |   point: 'assets/img/vest-point.webp',
45 |   tablet: 'assets/img/vest-tablet.webp',
46 |   cheer: 'assets/img/vest-cheer.webp'
47 | };
48 |
49 | /* Đổi tư thế nhân vật, có nhòe nhẹ để không giật hình. */
50 | function setPose(pose) {
51 |   var img = $('#tourHuong');
52 |   var src = POSE_SRC[pose || 'welcome'];
53 |   if (!img || !src || img.getAttribute('src') === src) return;
54 |   img.style.opacity = '0';
55 |   setTimeout(function () {
56 |     img.setAttribute('src', src);
57 |     img.style.opacity = '';
58 |   }, 180);
59 | }
60 |
61 | /* Giọng đọc thật do tác giả thu. Hai bộ giờ đủ cả 11 bước như nhau nên
62 | dùng chung một bảng tên tệp; thư mục ngôn ngữ mới là chỗ khác nhau.
63 | Nếu vì lý do gì tệp không tải được thì vẫn tự rơi về giọng máy để
64 | tour không đứt quãng. */
65 | var TRACKS = {
66 |   'tour.s0': 's0.mp3', 'tour.s1': 's1.mp3', 'tour.s2': 's2.mp3',
67 |   'tour.s3': 's3.mp3', 'tour.s4': 's4.mp3', 'tour.s5': 's5.mp3',
68 |   'tour.nav': 'nav.mp3', 'tour.navm': 'navm.mp3', 'tour.s6': 's6.mp3',
69 |   'tour.s7': 's7.mp3', 'tour.s8': 's8.mp3', 'tour.done': 'done.mp3'
70 | };
71 | var audio = null;
72 |
73 | function audioFor(key) {
74 |   var lang = global.I18n.lang;
75 |   return TRACKS[key] ? 'assets/audio/' + lang + '/' + TRACKS[key] : null;
76 | }
77 |
78 | function stopAudio(rewind) {
79 |   if (!audio) return;
80 |   audio.pause();
81 |   if (rewind) { try { audio.currentTime = 0; } catch (e) { /* bỏ qua */ } }
82 | }
83 |
84 | /* Phát lời cho một bước: ưu tiên giọng thu sẵn, không có thì đọc máy. */
85 | function playStep(key, text, langOverride, onEnd) {
86 |   var src = audioFor(key);
87 |   if (!src) { speak(text, langOverride, onEnd); return; }
```



```

88 |
89 |     stopSpeaking();
90 |     showVoiceNote(false);
91 |
92 |     if (!audio) audio = new Audio();
93 |     audio.onended = null;
94 |     audio.onerror = null;
95 |     audio.src = src;
96 |     audio.onended = function () { if (onEnd) onEnd(); };
97 |     // Tệp hổng hoặc chưa tải được thì quay về giọng máy chứ không im lặng
98 |     audio.onerror = function () { speak(text, langOverride, onEnd); };
99 |     var p = audio.play();
100 |     if (p && p.catch) p.catch(function () { speak(text, langOverride, onEnd); });
101 | }
102 |
103 | var index = 0;
104 | var running = false;
105 | var done = false;
106 | var paused = false;
107 | var steps = [];
108 | var spot = null;
109 | var synth = global.speechSynthesis || null;
110 | var lastVoiceLang = null;
111 |
112 | /* ----- Giọng đọc ----- */
113 |
114 | function voiceFor(lang) {
115 |     if (!synth) return null;
116 |     var want = (lang === 'en' || lang === 'fr') ? lang : 'vi';
117 |     var voices = synth.getVoices() || [];
118 |     var exact = voices.filter(function (v) {
119 |         return (v.lang || '').toLowerCase().indexOf(want) === 0;
120 |     });
121 |     if (!exact.length) return null;
122 |     // ưu tiên giọng cài sẵn trong máy để đọc được cả khi không có mạng
123 |     var local = exact.filter(function (v) { return v.localService; });
124 |     return (local[0] || exact[0]);
125 | }
126 |
127 | var FALLBACK_LANG = { en: 'en-US', vi: 'vi-VN', fr: 'fr-FR' };
128 |
129 | function speak(text, langOverride, onEnd) {
130 |     if (!synth) { showVoiceNote(true); return; }
131 |     synth.cancel();
132 |
133 |     var u = new SpeechSynthesisUtterance(text);
134 |     var lang = langOverride || global.I18n.lang;
135 |     var v = voiceFor(lang);
136 |     if (v) u.voice = v;
137 |     u.lang = v ? v.lang : (FALLBACK_LANG[lang] || 'vi-VN');
138 |     u.rate = lang === 'en' ? 0.98 : 1.0;
139 |     u.pitch = 1.05;
140 |     u.onend = function () { if (onEnd) onEnd(); };
141 |     u.onerror = function () { showVoiceNote(true); };
142 |
143 |     showVoiceNote(!v);
144 |     lastVoiceLang = lang;
145 |     synth.speak(u);
146 | }
147 |
148 | function stopSpeaking() {
149 |     if (synth) synth.cancel();

```

```
150 | }
151 |
152 | function stopAll(rewind) {
153 |     stopSpeaking();
154 |     stopAudio(rewind);
155 | }
156 |
157 | function showVoiceNote(on) {
158 |     var el = $('#tourNote');
159 |     if (!el) return;
160 |     el.textContent = on ? t('tour.noVoice') : '';
161 |     el.classList.toggle('hidden', !on);
162 | }
163 |
164 | /* ----- Vùng được làm nổi ----- */
165 |
166 | function clearSpot() {
167 |     if (spot) { spot.classList.remove('tour-spot'); spot = null; }
168 | }
169 |
170 | /* Kiểm tra phần tử có thực sự hiển thị hay không.
171 |    Không dùng offsetParent: phần tử position:fixed luôn trả về null dù đang
172 |    hiện rõ, khiến hai thanh điều hướng bị loại oan khỏi tour. */
173 | function isVisible(el) {
174 |     return !(el && el.getClientRects().length);
175 | }
176 |
177 | function highlight(sel) {
178 |     clearSpot();
179 |     if (!sel) return;
180 |     var el = document.querySelector(sel);
181 |     if (!isVisible(el)) return;
182 |     spot = el;
183 |     el.classList.add('tour-spot');
184 |     el.scrollIntoView({ behavior: 'smooth', block: 'center' });
185 | }
186 |
187 | /* ----- Bảng thuyết minh ----- */
188 |
189 | function panel() { return $('#tourPanel'); }
190 |
191 | function render() {
192 |     var step = steps[index];
193 |     $('#tourStep').textContent = t('tour.step', { i: index + 1, n: steps.length });
194 |     $('#tourText').textContent = t(step.key);
195 |     $('#tourWho').textContent = t('tour.name');
196 |
197 |     $('#tourPrev').disabled = index === 0;
198 |     $('#tourPrev').textContent = t('tour.prev');
199 |     $('#tourNext').textContent = index === steps.length - 1 ? t('tour.finish') : t('tour.next');
200 |     $('#tourPlay').textContent = paused ? t('tour.resume') : t('tour.play');
201 | }
202 |
203 | /* Thẻ chốt: đi hết tour thì mời vào làm bài thay vì đóng cái rụp. */
204 | function showDone() {
205 |     stopAll(true);
206 |     clearSpot();
207 |     done = true;
208 |     $('#tourSteps').classList.add('hidden');
209 |     $('#tourDone').classList.remove('hidden');
210 |     $('#tourStep').textContent = '';
211 |     setPose('cheer');
```

```
212 | // phần kết cũng có bản thu riêng
213 | playStep('tour.done', t('tour.done.lead'), null, null);
214 | }
215 |
216 | function hideDone() {
217 |     done = false;
218 |     $('#tourDone').classList.add('hidden');
219 |     $('#tourSteps').classList.remove('hidden');
220 | }
221 |
222 | function go(i) {
223 |     if (i < 0) return;
224 |     if (i >= steps.length) { showDone(); return; }
225 |     index = i;
226 |     paused = false;
227 |     render();
228 |     setPose(steps[index].pose);
229 |     highlight(steps[index].target);
230 |     playStep(steps[index].key, t(steps[index].key), steps[index].lang, function () {
231 |         // tự chuyển bước khi đọc xong, trừ bước cuối
232 |         if (running && !paused) {
233 |             setTimeout(function () { if (running && !paused && !done) go(index + 1); }, 700);
234 |         }
235 |     });
236 | }
237 |
238 | function start() {
239 |     steps = STEPS.filter(function (s) {
240 |         return isVisible(document.querySelector(s.target));
241 |     });
242 |     if (!steps.length) return;
243 |
244 |     running = true;
245 |     paused = false;
246 |     index = 0;
247 |     hideDone();
248 |     panel().classList.remove('hidden');
249 |     $('#tourFab').classList.add('hidden');
250 |     go(0);
251 | }
252 |
253 | function stop() {
254 |     running = false;
255 |     paused = false;
256 |     hideDone();
257 |     stopAll(true);
258 |     clearSpot();
259 |     var p = panel();
260 |     if (p) p.classList.add('hidden');
261 |     $('#tourFab').classList.remove('hidden');
262 | }
263 |
264 | function togglePlay() {
265 |     if (!running) return;
266 |     if (paused) {
267 |         paused = false;
268 |         if (audio && audio.src && !audio.ended && audio.currentTime > 0) {
269 |             audio.play();
270 |             render();
271 |         } else {
272 |             go(index);
273 |         }
```

```
274 |     } else {
275 |         paused = true;
276 |         stopAll(false);
277 |         render();
278 |     }
279 | }
280 |
281 | /* ----- Khởi tạo ----- */
282 |
283 | var Tour = {
284 |     start: start,
285 |     stop: stop,
286 |     get running() { return running; },
287 |
288 |     /** Dịch lại nhãn khi người dùng đổi ngôn ngữ giữa chừng. */
289 |     refresh: function () {
290 |         $('#tourFabLabel').textContent = t('tour.start');
291 |         if (running && !done) {
292 |             render();
293 |             // đọc lại bước hiện tại bằng ngôn ngữ mới
294 |             if (!paused) playStep(steps[index].key, t(steps[index].key), steps[index].lang);
295 |         }
296 |     },
297 |
298 |     init: function () {
299 |         $('#tourFabLabel').textContent = t('tour.start');
300 |         $('#tourFab').addEventListener('click', start);
301 |         $('#tourClose').addEventListener('click', stop);
302 |         $('#tourPrev').addEventListener('click', function () { go(index - 1); });
303 |         $('#tourNext').addEventListener('click', function () { go(index + 1); });
304 |         $('#tourPlay').addEventListener('click', togglePlay);
305 |
306 |         // "Bắt đầu làm bài": đóng tour rồi mở thẳng màn thiết lập đề thi
307 |         $('#tourGo').addEventListener('click', function () {
308 |             stop();
309 |             var card = document.querySelector('.mode-card[data-mode="exam"]');
310 |             if (card) card.click();
311 |         });
312 |         $('#tourAgain').addEventListener('click', function () { hideDone(); start(); });
313 |
314 |         // danh sách giọng đọc thường nạp không đồng bộ
315 |         if (synth && typeof synth.onvoiceschanged !== 'undefined') {
316 |             synth.onvoiceschanged = function () {
317 |                 if (running && !paused) showVoiceNote(!voiceFor(lastVoiceLang || global.I18n.lang));
318 |             };
319 |         }
320 |
321 |         document.addEventListener('keydown', function (e) {
322 |             if (!running) return;
323 |             if (e.key === 'Escape') stop();
324 |         });
325 |
326 |         // dừng đọc khi rời trang để giọng không còn vang sau khi đóng tab
327 |         global.addEventListener('pagehide', function () { stopAll(true); });
328 |         global.addEventListener('beforeunload', function () { stopAll(true); });
329 |     }
330 | };
331 |
332 | global.Tour = Tour;
333 | })(window);
```